AGENT EVALUATION
SCIENCE

AFP-Enterprise: Evaluating Host-Level Runtime Containment for Planner–Tool Execution

Muchen He[1,*]

[1] Independent Researcher

[*] Correspondence to Muchen He.

Abstract

AFP-Enterprise is a host runtime kernel beside a local planner, reached by a synchronous pre-flight probe. Before irreversible tool I/O, the planner reports recursion depth and context, then blocks on allow, delay, or isolate. The kernel does not parse prompts or tool schemas. It checks, in order, an emergency stop, a depth cap, a four-axis pressure maximum, and a specified isolate–recover state machine. On a LangGraph planner with real lookup and HTTP GET tools, five bounded jobs complete under no gate, a per-step wipe, and the current sidecar (false isolate $= 0$). A scripted loop and a 48-GET burst complete without a gate and are contained by both gated variants. Those traces fire the depth cap and the hard pressure isolate; they do not reach the delay band or the recover machine. The two gated variants match on those jobs. They differ only when a later probe reuses leftover host telemetry: AFP-Enterprise refuses; the wipe allows. A new job that reports depth 0 is admitted under both. Two planners on one sidecar can overwrite each other, so the kernel keeps last-reported host telemetry, not a lock on who the planner is.

Keywords: Planner–tool runtime; Admission control; Pre-flight check; Retained telemetry; Stale-state retention; Circuit breaker; Multi-agent systems

1 Introduction

Deployed agents now decompose work, call tools, and talk to other agents (Yao et al., 2023; Wu et al., 2023; LangChain, 2025; Anthropic, 2024; Google, 2025). Safety evaluations have followed injection, tool misuse, and harmfulness (Greshake et al., 2023; Debenedetti et al., 2024; Andriushchenko et al., 2024; Hou et al., 2025). Those tests are necessary and incomplete. A planner can fan out into $A \to B \to C$, sometimes closing a loop, while every hop remains a valid tool call. The trajectory still becomes physically unsustainable: depth, context, concurrency, and burst grow until the node saturates.

That failure is easy to miss. Ordinary admission control makes a per-request decision from current resource state and forgets when the request ends. Token budgets often account after commit. Framework retry and recursion limits reset when the session ends (Cemri et al., 2025). MCP and A2A decide *what* may be said (Anthropic, 2024; Google, 2025), not *whether* the next hop may execute. If the harness only records task success or policy violations, a graph that loops itself into unavailability can still look well-behaved.

This paper evaluates **AFP-Enterprise** on a planner–tool harness. The object is planner–tool execution, not a multi-agent mesh. Before a local planner emits a tool call—a lookup, an HTTP GET, or an MCP tool—it asks a sidecar whether that call may run. The planner and tools are unchanged; the sidecar admits, delays, or refuses the next hop. The proposed distinction versus ordinary admission control is not a stronger first-time breaker. It is retaining execution telemetry across request boundaries and feeding that state into later pre-flight decisions.

Three questions:

1. Can a pre-flight gate contain the tested loop and burst without blocking bounded work?
2. Does retained host telemetry add behavior beyond a per-step wipe?
3. What is the state-ownership boundary on a shared sidecar?

On the jobs that were tested: yes; only on a stale probe; host-level, and a later planner can overwrite it. The delay band and the isolate–recover machine are specified and were not reached. These traces do not establish multi-agent or production benefits.

2 What AFP-Enterprise is

2.1 Planner interface

The planner does not embed the kernel. Before a tool call it writes telemetry, then blocks on `PreFlightCheck`. The sidecar does not parse prompts or tool schemas. Each hop is write-then-probe:

1. report recursion depth and context bytes;
2. probe with a burst hint (estimated imminent fan-out);
3. proceed only on allow; sleep on delay; halt I/O on isolate.

Pre-flight is the call. The logic that answers it sits in the host kernel.

2.2 Host kernel

The kernel evaluates four stages in this order and returns on the first match.

1. *Emergency stop.* An operator kill switch isolates every hop.
2. *Depth cap.* Reported depth > 10 isolates immediately.
3. *Pressure isolate.* Pressure is a maximum, not an average,

$$e = \max\big(e_{\text{tool}},\, e_{\text{mem}},\, e_{\text{ctx}},\, e_{\text{burst}}\big), \tag{1}$$

   with $e_{\text{tool}} = n_{\text{active}}/50$, $e_{\text{mem}} = \text{mem}/0.90$, $e_{\text{ctx}} = \text{bytes}/\texttt{MaxContextBytes}$, and $e_{\text{burst}} = \texttt{estimated_tasks}/50$. Each term is capped at 1. The storm job's hint of 48 gives $e_{\text{burst}} = 0.96$. A hard isolate fires at $e^\star = 0.95$.
4. *Isolate–recover machine.* If the first three stages pass, a specified machine may allow, delay, isolate, or admit a low-rate probe. Delay starts above $e_{\text{warn}} = 0.75$ (500–2000 ms). Allow resumes below $e_{\text{safe}} = 0.40$. Isolate holds for a minimum dwell (64 epochs), then probation (128 epochs) before allow; pressure above the safe band during probation re-isolates.

These traces fire stage 2 (the loop) or stage 3 (the burst). They do not trip the emergency stop, the delay band, or the recover machine. Depth and hard isolate return before that machine is consulted.

2.3 What the kernel keeps

Last-reported host telemetry—here, recursion depth—stays until a later report overwrites it. Request completion does not clear it. That is stale-state retention, not a lock on agent identity. State is host-level: two planners on one sidecar share one depth slot.

2.4 What we compare

The graph and tools are identical across conditions. *No gate:* the planner and tools always run; the only stop is a graph depth cap. *Per-step wipe:* the same AFP-Enterprise checks run, but each request starts by reporting depth 0, clearing leftover telemetry. *Current AFP-Enterprise sidecar:* leftover depth is kept until a later report overwrites it.

A gate-versus-off table is not enough to say what AFP-Enterprise adds. The wipe is the relevant gated baseline: it already contains the tested loop and burst.

What these traces do not measure: prompt-injection robustness, computer-use safety, MCP ingress, multi-agent isolation, production throughput, or LLM-agent competence. A local Qwen2.5 0.5B arm is supplementary (Appendix B). A 500-node Monte Carlo is a model invariant (Appendix E).

3 Related work

Agent safety benchmarks grade harmfulness, injection, and tool abuse (Debenedetti et al., 2024; Andriushchenko et al., 2024; Ruan et al., 2024; Greshake et al., 2023). Control and oversight constrain intent (Greenblatt et al., 2024; Zou et al., 2024; Kenton et al., 2024). Frameworks optimize task completion (Yao et al., 2023; Wu et al., 2023; LangChain, 2025). Admission control is standard operations practice (Nygard, 2018; Fowler, 2014; Envoy Proxy, 2024). The gap is not the existence of that primitive. Ordinary admission control decides from current resource state and drops that state when the request ends. The proposed distinction is retaining execution telemetry across request boundaries and using it on later pre-flight decisions. These traces demonstrate that distinction only under a stale-probe condition. They do not establish broader multi-agent or production-level benefits.

4 Method

4.1 Harness

All primary runs used one personal computer, one LangGraph, and two real tools: a local key lookup and HTTP GET against a loopback fixture. Pre-flight sits before the tool runs. The tables use a deterministic tool-calling policy so stacking and burst do not vary across repeats. A live OpenAI-compatible arm exists; it is not the primary evidence. Sidecar policy uses recursion cap 10 and concurrency cap 50. The unit of analysis is one job under one gate. The planner is deterministic, so repeats are a reproduction check, not a sampling distribution. That isolates sidecar admission from stochastic LLM routing; the live 0.5B arm is recorded separately in Appendix B. No unfinished row is dropped.

4.2 Jobs

S1 is one lookup: the shortest live hop. *S2* is eight hops (four lookups, four GETs), longer than S1 and still under the depth cap—the false-isolate check that a one-hop job does not cover. *S3* is graph-driven `tick` stacking to a cap of 40. *S4* is one `expand` with burst hint 48 ($e_{\text{burst}} = 48/50 = 0.96$), then queued GETs if allowed. Without a gate that is one expand plus 48 GETs—49 tool executions. Three extra bounded jobs (lookups only, fetches only, and eight repeats of one lookup) complete the five-job reliability set. After S3 isolate, a stale probe that does not rewrite depth, and a fresh S1 that reports depth 0, test leftover telemetry. Two planners on split sidecars versus one shared sidecar test state ownership.

Figure 1 summarizes the comparison. Conditions are assigned to the same graph; only leftover-state handling changes.

5 Results

5.1 Reliability and first-time containment

Table 1 reports no-gate versus current sidecar on the four named jobs. S1 and S2 finish with the gate on (false isolate $= 0$ on those jobs). S3 without the gate walks to the graph cap (depth 40, 39 ticks); with it, isolation fires at reported depth 11. S4 without the gate performs the expand plus 48 GETs (49 tool executions); with it, the burst hint opens the breaker at entropy 0.96 and no GET runs.

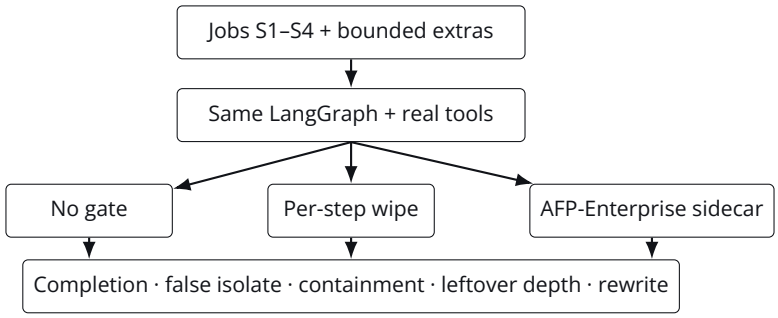


Figure 1 | Same graph and tools. The three conditions differ only in whether leftover host telemetry is absent, wiped, or retained by AFP-Enterprise.

Table 1 | No-gate versus current sidecar on real tool I/O. S4 off, Tools $= 49$: one expand plus 48 GETs. A per-step wipe matches the sidecar on these four jobs (Table 2).

Job	Gate	Done	Tools	Probes	Isolated	Depth	Reason
S1 short	off	yes	1	0	no	2	—
S1 short	on	yes	1	3	no	2	—
S2 long	off	yes	8	0	no	9	—
S2 long	on	yes	8	17	no	9	—
S3 runaway	off	to cap	39	0	no	40	—
S3 runaway	on	no	10	22	yes	11	recursion cap
S4 storm	off	yes	49	0	no	50	—
S4 storm	on	no	0	2	yes	1	burst entropy

Bounded completion extends to five jobs: S1, S2, lookups only, fetches only, and eight repeats of one lookup. All five complete under no gate, the wipe, and the current sidecar. False isolate on that set is 0. In these local controlled runs, mean wall-clock on the five bounded jobs was 3.8 ms off and 6.0–6.4 ms gated. That measurement is indicative only. It is not a throughput or scalability benchmark.

5.2 Where the gated variants differ

Table 2 is the result that a gate-versus-off table conceals. S1–S4 do not distinguish the two gated variants. Both contain the loop and the burst; neither false-isolates the bounded set. They differ only when a later execution reuses stale, uncleared telemetry: the current sidecar refuses; the wipe allows. A new S1 that reports depth 0 completes under both. Appendix D lists every job under all three gates from the latest three-way run, including lookups only, fetches only, and eight repeats.

Table 2 | Three-way baseline. Bounded jobs complete under every gate. The gated columns differ only on a stale post-isolate probe.

Scene	No gate	Per-step wipe	Current sidecar
Five bounded jobs	done	done	done
False isolate	0	0	0
Infinite loop / 48-burst	not stopped	stopped	stopped
Stale probe, no rewrite	—	allow	**refuse**
New job reports depth 0	—	allow	allow

This is evidence for leftover telemetry, not for a penalty locked to agent identity. After S3 isolate, a probe that does not rewrite depth is still refused by the current sidecar; a new graph that reports depth 0 is admitted. Request completion alone does not clear leftover depth. A later report of depth 0 does.

State ownership fails on a shared sidecar. Two planners on separate sidecars keep the second planner healthy while the first isolates on S3. On one shared sidecar, the second planner's fresh report overwrites the first planner's leftover depth, and the second planner completes S1 (Appendix A). Isolation is a host-level check, not a per-agent record. If two planners must not affect each other, they need separate sidecars, or the kernel must account per identity first.

A local Qwen2.5 0.5B completes S1 and does not finish S2 (Appendix B). That is a model-quality result, not LLM-agent validation. The scripted planner remains the primary evidence.

6 Limitations

The evidence is a controlled planner–tool environment on one machine. It is not computer-use, MCP ingress, injection robustness, multi-agent isolation, or production safety. The primary planner is scripted; the live 0.5B arm does not finish the long recipe. Jobs are deterministic, so the tables have no confidence intervals. Latency is indicative only. State ownership fails on a shared sidecar. The emergency stop, the delay band, and the isolate–recover machine were not reached. These traces stop a false persistence claim; they are not a complete runtime-safety benchmark.

7 Conclusion

Ordinary admission control decides from current resource state and forgets at request end. AFP-Enterprise's measured distinction is leftover host telemetry on later pre-flight probes. On the jobs that were tested, bounded work can finish, and a loop or burst can be stopped by the depth cap or the hard pressure isolate; a per-step wipe can do the same. The gated variants differ only on a stale probe. The delay band and the isolate–recover machine are specified; they are not evidenced by these tables. That is the claim—not production safety, not multi-agent isolation, and not a lock on agent identity.

Ethics Statement

This work records AFP-Enterprise on scripted and small-model planner–tool jobs. It does not involve human subjects, user data, or production traffic. The paper reports negative and bounding results (no difference on S1–S4 between the gated variants; failed state ownership; mesh 100% as an invariant) so that the traces cannot be read as field safety.

References

Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, et al. 2024. AgentHarm: A benchmark for measuring harmfulness of LLM agents. *arXiv preprint arXiv:2410.09024*. Cited at: 1, 2.

Anthropic. Model context protocol. https://modelcontextprotocol.io, 2024. Cited at: 1, 2.

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, et al. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*. Cited at: 1.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, 2024. Cited at: 1, 2.

Envoy Proxy. Circuit breaking. https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/circuit_breaking, 2024. Cited at: 1.

Martin Fowler. Circuitbreaker. https://martinfowler.com/bliki/CircuitBreaker.html, 2014. Cited at: 1.

Google. Agent2agent protocol (A2A). https://a2a-protocol.org, 2025. Cited at: 1, 2.

Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *International Conference on Learning Representations*, 2024. Cited at: 1.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023. Cited at: 1, 2.

Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model context protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*. Cited at: 1.

Zachary Kenton, Noah Siegel, János Kramár, Jonah Brown-Cohen, Samuel Albanie, Jannis Bulian, et al. 2024. On scalable oversight with weak LLMs judging strong LLMs. *arXiv preprint arXiv:2407.04622*. Cited at: 1.

LangChain. LangGraph. https://langchain-ai.github.io/langgraph/, 2025. Cited at: 1, 2.

Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2 edition, 2018. Cited at: 1.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, et al. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024. Cited at: 1.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*. Cited at: 1, 2.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. Cited at: 1, 2.

Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, et al. Improving alignment and robustness with circuit breakers. In *Advances in Neural Information Processing Systems*, 2024. Cited at: 1.

A Stale telemetry and two planners

Isolation follows last-reported host telemetry on one sidecar, not a lock on agent identity. Table A1 is the rewrite check after S3 isolate. Table A2 is the ownership check.

Table A1 | Same sidecar after S3 isolate. A stale probe keeps leftover depth 11; a new S1 reports 0 and overwrites it.

Arm	Isolated	S1 done	Leftover depth	Reason
Stale probe (no rewrite)	yes	—	11	recursion cap
Fresh S1 (reports depth 0)	no	yes	0	—

Table A2 | Two local planners. Split sidecars keep B healthy; a shared sidecar lets B overwrite A's telemetry. A's S3 is 10 tools / 22 probes; B's S1 is 1 tool / 3 probes in both layouts.

Layout	Agent	Job	Isolated	Done
Split sidecars	A	S3	yes	no
Split sidecars	B	S1	no	yes
Shared sidecar	A	S3	yes	no
Shared sidecar	B	S1	no	yes

B Live run on a local small model

Table B1 uses Qwen2.5 0.5B via a local OpenAI-compatible endpoint, the same graph, and the same tools. S1 is the only completed live job (604 prompt tokens in both modes). S2 without the gate hits LangGraph `recursion_limit`$= 30$ with 0 tools and $18{,}859$ prompt tokens. S2 with the gate isolates a looping planner at depth 11 (10 tools, $10{,}799$ prompt tokens). Neither cell is a false isolate of a finished eight-tool job. Live wall-clock is model-bound and is not used as an overhead figure.

Table B1 | Live AFP-Enterprise on Qwen2.5 0.5B. Supplementary only.

Job	Gate	Done	Tools	Isolated	Tokens	Reason
S1 short	off	yes	1	no	604	—
S1 short	on	yes	1	no	604	—
S2 long	off	no	0	no	$18{,}859$	graph `recursion_limit`$= 30$
S2 long	on	no	10	yes	$10{,}799$	recursion cap

C Depth-only unit check

Before tools were wired, the same sidecar isolated a depth-only planner (no lookup, no HTTP) at the recursion cap. Table C1 is that check; Table 1 is the main tool result.

D Three-way baseline, full latest run

Table D1 restates Table 2 by gate. Table D2 is the latest three-way CSV: every bounded job, S3, S4, and the two post-isolate probes, under all three gates. Mean wall-clock on the five bounded jobs is 3.8 ms off, 6.4 ms wipe, and 6.0 ms current sidecar; indicative only.

Table C1 | Depth-only LangGraph (no tool I/O). Unit check, not the main evidence.

Job	Gate	Finished	Steps	Probes	Isolated	Depth	Reason
Benign	off	yes	3	0	no	3	—
Benign	on	yes	3	3	no	3	—
Runaway	off	to cap	40	0	no	40	—
Runaway	on	no	11	12	yes	11	recursion cap

Table D1 | Three-way baseline by gate. Bounded jobs: $5/5$ complete, false isolate $= 0$.

Gate	Bounded done	S3/S4 isolated	Stale probe	Fresh S1
Off	$5/5$	$0/2$	—	—
Per-step wipe	$5/5$	$2/2$	allow	done
Current sidecar	$5/5$	$2/2$	refuse	done

Table D2 | Latest three-way run (scripted planner, real tools). Wipe matches the current sidecar on every job row; they differ only on the stale probe.

Job	Gate	Done	Tools	Probes	Isolated	Wall (ms)	Reason
S1 short	off	yes	1	0	no	1.6	—
S1 short	wipe	yes	1	3	no	2.0	—
S1 short	sidecar	yes	1	3	no	2.0	—
S2 long	off	yes	8	0	no	8.3	—
S2 long	wipe	yes	8	17	no	9.4	—
S2 long	sidecar	yes	8	17	no	8.7	—
Lookups only	off	yes	4	0	no	1.5	—
Lookups only	wipe	yes	4	9	no	3.8	—
Lookups only	sidecar	yes	4	9	no	4.2	—
Fetches only	off	yes	6	0	no	5.3	—
Fetches only	wipe	yes	6	13	no	10.3	—
Fetches only	sidecar	yes	6	13	no	8.4	—
Eight repeats	off	yes	8	0	no	2.1	—
Eight repeats	wipe	yes	8	17	no	6.3	—
Eight repeats	sidecar	yes	8	17	no	6.7	—
S3 runaway	off	yes	39	0	no	8.3	—
S3 runaway	wipe	no	10	22	yes	7.9	recursion cap
S3 runaway	sidecar	no	10	22	yes	8.1	recursion cap
S4 storm	off	yes	49	0	no	30.8	—
S4 storm	wipe	no	0	2	yes	1.5	burst entropy
S4 storm	sidecar	no	0	2	yes	1.9	burst entropy
Stale probe	wipe	—	0	1	no	—	allow after wipe
Stale probe	sidecar	—	0	1	yes	—	recursion cap
Fresh S1	wipe	yes	1	3	no	—	reports depth 0
Fresh S1	sidecar	yes	1	3	no	—	reports depth 0

E Synthetic mesh: an invariant, not a field result

Death is defined as load ≥ 1. The simulated gate refuses any request with load + request > 0.95. Under that pair of rules a node cannot die: 100% survival is an invariant of the construction. The complementary fact is that without any pre-accumulation refusal, the same seeds collapse (mean $1.876/500$ alive at $T = 100$ over $1{,}000$ runs). Viability v is a floor used only in this simulated mesh; it is not part of the LangGraph sidecar traces. Table E1 locates the invariant in the breaker, not in v. Tables E2 and E3 are the same invariant under a malicious-rate sweep and a reduced per-request load. The 100% cells are an invariant of the construction, not a field result.

Table E1 | Mean survival (%) at $T = 100$. Breaker-only matches the full gate; viability-only matches baseline.

Malicious %	Full	No v floor	No breaker	No burst tax	Breaker only	v only / base
5	100	100	4.8	100	100	0.3 / 0.3
20	100	100	15.7	100	100	0.2 / 0.2

Table E2 | Mean survival (%) at $T = 100$ versus malicious fraction (300 runs). Every full-gate cell is the breaker invariant.

Mode	1%	5%	10%	15%	20%	30%	40%	50%
Baseline	2.3	0.3	0.3	0.3	0.2	0.2	0.2	0.2
Full gate	100	100	100	100	100	100	100	100

Table E3 | Mean survival (%) when malicious per-request load is reduced (5% malicious, 300 runs).

Malicious load	0.80	0.45	0.20	0.12
Baseline	0.3	0.6	3.1	7.8
Full gate	100	100	100	100